

Singable Lyrics Translation with IPA-Augmented LLM Agents

Anonymous ACL submission

Abstract

Translating song lyrics while preserving singability requires satisfying musical constraints—syllable counts, rhyme schemes, and syllable patterns—that standard machine translation ignores. We present a framework that equips a large language model with IPA-based phonetic analysis tools within a ReAct-style agent loop. The agent extracts constraints from source lyrics, then translates through a multi-phase process, calling tools to iteratively verify and refine each constraint. We formalize the three musical constraints, describe the IPA tool suite and agent architecture, and propose three evaluation metrics: Syllable Error Rate (SER), Syllable Count Relative Error (SCRE), and Adjusted Rand Index (ARI).

1 Introduction

A translated song lyric must conform to the musical structure of the original so that it remains *singable* to the same melody (Low, 2005). This requires satisfying hard structural constraints: syllable counts must match melodic phrases, rhyme schemes should be preserved, and word-level syllable distributions should mirror the original phrasing. Despite theoretical grounding (Low, 2005; Franzon, 2008), computational approaches remain scarce. Neural MT systems have no mechanism for phonetic constraints, and constrained decoding methods (Hokamp and Liu, 2017; Post and Vilar, 2018) address lexical but not phonetic constraints. A fundamental obstacle is that *counting syllables, detecting rhyme, and analyzing phonetic patterns are not tasks that LLMs perform reliably through text generation alone* (Gao et al., 2023).

We address this by equipping an LLM with IPA-based phonetic tools within a ReAct-style agent loop (Yao et al., 2023), enabling iterative, constraint-aware translation refinement. Our framework is open-source and available as a PyPI pack-

age for reproducibility and practical use.¹

Our contributions are:

1. A formalization of three musical constraints—syllable counts, rhyme schemes, and syllable patterns—and IPA-based methods for their extraction and verification (§3).
2. An IPA-augmented tool suite and multi-phase agent loop architecture for constraint-preserving lyrics translation (§4).
3. Three automatic evaluation metrics—SER, SCRE, and ARI—designed to measure structural constraint preservation in singable translation (§5).

2 Related Work

2.1 Song Translation

Low (2005) proposed treating song translation as a pentathlon balancing singability, sense, naturalness, rhythm, and rhyme, observing that rhythm is often the hardest constraint. Franzon (2008) identified five translation strategies ranging from adapting lyrics to rewriting both lyrics and melody. Low (2022) provided an updated theoretical account reinforcing singability as the guiding criterion.

On the computational side, Ou et al. (2023a) formalized singable lyric translation as a constrained sequence-to-sequence problem, fine-tuning a neural MT model with length, rhyme, and word boundary constraints for English–Chinese translation. Guo et al. (2022) addressed automatic song translation for tonal languages, introducing tonal constraints beyond syllable matching, and Ghazvininejad et al. (2018) tackled neural poetry translation with rhythm and rhyme constraints using constrained decoding. Ye et al. (2024) proposed

¹Code and package available at <https://github.com/anonymous> (anonymized for review).

quality-aware musical lyrics translation that explicitly models musical features during generation. Ren et al. (2025) introduced a curriculum reinforcement learning framework for controllable lyric translation, balancing multiple constraints through difficulty-aware training. Zhao et al. (2025) addressed melody-constrained lyrics editing, and Ou et al. (2023b) enforced structural wording and formatting constraints in melody-to-lyric generation. Cho et al. (2025) introduced MAVL, a multilingual audio-video lyrics dataset, and Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation. Beyond translation, Chen and Teufel (2024) used scansion to generate Mandarin lyrics matching melodies, Liang et al. (2024) improved singability of AI-generated lyrics through prosody explanations, Yu et al. (2025) modeled lyric-melody alignment for generation, and Yoo et al. (2025) demonstrated that singability challenges extend to multimodal settings such as sign language. These approaches rely on specialized model fine-tuning, constrained decoding, or reinforcement learning; our work instead uses a general-purpose LLM augmented with IPA verification tools in an agent loop, requiring no task-specific training.

2.2 Constrained Text Generation

Hokamp and Liu (2017) and Post and Vilar (2018) introduced lexically constrained decoding via Grid Beam Search and Dynamic Beam Allocation. In computational poetry, Ghazvininejad et al. (2016) used finite-state machinery for meter and rhyme, Hopkins and Kiela (2017) generated rhythmic verse with neural networks, and Lau et al. (2018) jointly modeled poetic language, meter, and rhyme. These approaches enforce constraints at the decoding level or learn them implicitly; ours uses an agent loop where the LLM generates freely but verifies via external tools.

2.3 Tool-Augmented Language Models

Recent work has shown that LLMs can be made substantially more capable by giving them access to external tools. Schick et al. (2023) trained language models to autonomously decide when to call tools such as calculators and search engines. ReAct (Yao et al., 2023) introduced a paradigm where LLMs alternate between generating reasoning traces and performing actions via external tools, improving performance on tasks requiring grounded, multi-step decision-making. PAL (Gao et al., 2023)

demonstrated that offloading computational steps to program execution significantly improves accuracy on reasoning tasks, a finding directly relevant to our use case: syllable counting is a computational task that LLMs perform unreliably but that IPA-based tools handle deterministically. Chain-of-thought prompting (Wei et al., 2022) provides a complementary mechanism for structured reasoning that we combine with tool use in our agent loop. In machine translation, Jiao et al. (2023) demonstrated that LLMs achieve strong translation quality, but their evaluation did not consider musical or structural constraints.

2.4 Phonetic Resources

Our tool suite builds on Phonemizer (Bernard and Titeux, 2021) for text-to-IPA conversion and Pan-Phon (Mortensen et al., 2016) for phonetic distance computation; both are described in §4.

3 Musical Constraints in Lyrics Translation

A singable translation must satisfy three structural constraints, each arising from a different aspect of the relationship between lyrics and melody. Below we describe each constraint, formalize it, and explain how we extract and verify it using IPA-based analysis.

3.1 Syllable Count Constraint

In sung performance, each syllable maps to one or more musical notes. When a melody assigns three notes to a phrase, the lyric must contain exactly three syllables—fewer leave notes unmatched, while extra syllables force cramming that disrupts the rhythmic feel. This makes syllable count the most rigid of the three constraints: rhyme and phrasing mismatches may be tolerable in some styles, but syllable count mismatches are almost always audible. Formally, given source lyrics $L_s = (l_1, \dots, l_n)$ in language λ_s , the constraint requires that translated lyrics $L_t = (l'_1, \dots, l'_n)$ in target language λ_t satisfy $\text{syll}(l'_i, \lambda_t) = \text{syll}(l_i, \lambda_s)$ for all lines i .

Syllable counting is language-dependent. For Chinese, each character constitutes exactly one syllable, so counting is trivial. For other languages, we convert text to IPA via Phonemizer (Bernard and Titeux, 2021) and count *vowel nuclei*—the vocalic centers of syllables:

$$\text{syll}(l_i, \lambda_s) = |\text{nuclei}(\text{IPA}(l_i, \lambda_s))| \quad (1)$$

where $\text{nuclei}(\cdot)$ matches a predefined set of IPA vowel symbols and diphthongs (e.g., a, i, u, ə, ai, ei, ou), with adjacent vowels forming diphthongs counted as single nuclei. For example, “*Let it go*” yields IPA /lət it ɡəʊ/ with three vowel nuclei [ɛ, ɪ, əʊ], giving a count of 3. The resulting sequence $\mathbf{s} = (s_1, \dots, s_n)$ serves as the hard constraint for translation.

3.2 Rhyme Scheme Constraint

Rhyme provides structural cohesion in lyrics, creating sonic expectations that are satisfying when met and jarring when violated. A song with an *AABB* scheme feels fundamentally different from one with *ABAB*; preserving the rhyme scheme ensures the translation maintains the same pattern of sonic echoes as the original. Formally, a rhyme scheme is a labeling $R = (r_1, \dots, r_n)$ where $r_i \in \{A, B, C, \dots\}$ assigns each line to a rhyme class, with $r_i = r_j$ if and only if lines i and j rhyme. The constraint requires that the translation’s scheme R_t match the source scheme R_s .

We detect rhyme by comparing phonetic endings rather than orthographic forms. For each line, we extract the *rhyme ending*: the IPA substring from the last vowel nucleus to the end of the transcription. For Chinese, we extract the pinyin final of the last character instead. The scheme is constructed by labeling the first line *A*, and assigning each subsequent line the label of any earlier line it rhymes with, or a new label otherwise. Consider the following example:

*The stars shine bright with silver **light*** → /laɪt/ → ending: /aɪt/
*And covers all in purest **white*** → /waɪt/ → ending: /aɪt/
*I stand and watch the world **below*** → /bɪləʊ/ → ending: /əʊ/
*As gentle winds begin to **blow*** → /bləʊ/ → ending: /əʊ/

Lines 1–2 share ending /aɪt/ (class *A*) and lines 3–4 share /əʊ/ (class *B*), yielding *AABB*. This IPA-based approach correctly identifies rhymes that orthographic comparison would miss (e.g., “weight” and “late”) and rejects false rhymes based on spelling alone (e.g., “cough” and “through”).

3.3 Syllable Pattern Constraint

Even when two lines share the same total syllable count, different word-level distributions create different rhythmic feels. Consider two eight-syllable lines: “un-for-get-ta-ble mel-o-dy” with pattern [5, 3] versus “I can hear the mu-sic play”

with pattern [1, 1, 1, 1, 2, 2]. The first has two long words producing a flowing feel, while the second has many short words producing a staccato rhythm. Word boundaries interact with note groupings and breaths in sung performance, so preserving the syllable pattern ensures the translation fits the melody at the phrasing level, not just the syllable level. Formally, the syllable pattern of a line l_i is a vector $\mathbf{p}_i = (p_{i,1}, \dots, p_{i,k_i})$ recording the syllable count of each word. The constraint asks that the translation’s pattern $\mathbf{p}'_i$ be similar to $\mathbf{p}_i$ while maintaining the same total: $\sum_j p'_{i,j} = \sum_j p_{i,j} = s_i$.

To extract patterns, we apply language-specific word segmentation—space-based tokenization for English and other space-delimited languages, and a neural tokenizer for Chinese—then compute each word’s syllable count via Equation 1. We measure pattern similarity using a normalized alignment score. Given patterns of potentially different lengths, we pad the shorter one with zeros and compute:

$$\text{sim}(\mathbf{p}, \mathbf{p}') = 1 - \frac{\sum_{j=1}^m |p_j - p'_j|}{\sum_{j=1}^m \max(p_j, p'_j)} \quad (2)$$

where $m = \max(|\mathbf{p}|, |\mathbf{p}'|)$. A score of 1.0 indicates an exact match; we treat ≥ 0.8 as acceptable. For example, “Let it go” has pattern [1, 1, 1]. A Chinese translation tokenized as two words with pattern [2, 1] yields $\text{sim}([1, 1, 1], [2, 1, 0]) = 0.5$ (poor alignment), whereas a translation tokenized as three single-character words gives pattern [1, 1, 1] with $\text{sim} = 1.0$.

4 IPA-Augmented Agent Framework

We now describe the agent architecture that uses IPA tools to produce constraint-preserving translations. Figure 1 provides an overview of the complete pipeline.

4.1 IPA as Universal Phonetic Interface

The International Phonetic Alphabet provides a standardized, language-independent representation of speech sounds, making it an ideal intermediary for cross-lingual phonetic analysis: regardless of the source or target language, all phonetic operations—syllable counting, rhyme comparison, pattern analysis—can be performed on IPA transcriptions using the same algorithms. Our system uses Phonemizer (Bernard and Titeux, 2021) as the text-to-IPA backend, wrapping the eSpeak NG

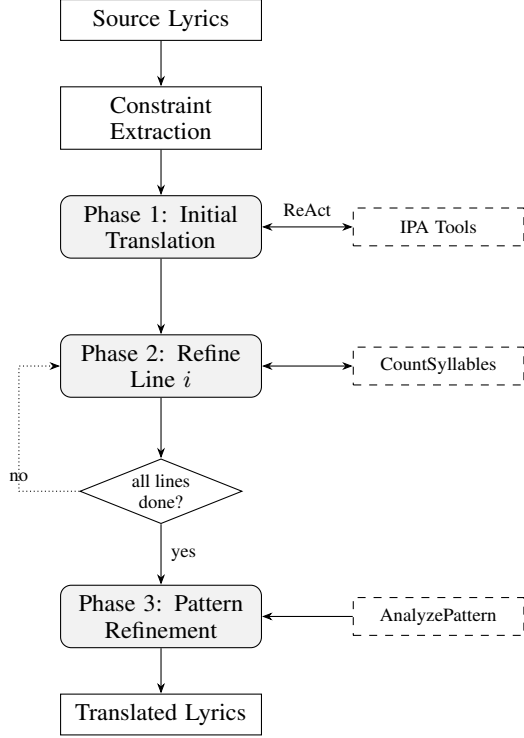


Figure 1: Agent framework overview. Phase 1 translates all lines using an internal ReAct loop with all IPA tools. Phase 2 refines each line sequentially (up to 10 attempts per line); the dotted arrow shows the graph-level loop that iterates through lines. Phase 3 is a single pass that uses ANALYZEPATTERN output to guide pattern refinement without a retry loop.

speech synthesizer with support for over 100 languages. Language codes are normalized internally (e.g., zh, zh-cn, cmn all map to Mandarin Chinese) to provide a uniform interface. For phonetic distance computation, we use PanPhon (Mortensen et al., 2016), which maps each IPA segment to a vector of articulatory features, enabling fine-grained similarity measurement.

4.2 Tool Suite

The agent has access to four core IPA-based tools (Table 1), each wrapping a deterministic function that provides capabilities the LLM cannot reliably perform through text generation alone (Gao et al., 2023).

COUNTSYLLABLES is the most frequently called tool: given a text string and language code, it converts to IPA and returns an integer syllable count (§3.1). TEXTTOIPA exposes the raw IPA transcription, allowing the agent to inspect phonetic realizations and reason about which syllables to modify—a form of chain-of-thought reasoning (Wei et al., 2022) grounded in phonetic observa-

Tool	Description
COUNTSYLLABLES	Count syllables via IPA vowel nuclei (or character count for Chinese)
TEXTTOIPA	Convert text to IPA transcription via eSpeak NG
DETECTRHYME	Detect the rhyme scheme of a set of lines by comparing IPA endings
ANALYZEPATTERN	Compare target vs. current syllable patterns with structured feedback

Table 1: Core IPA-based tools available to the agent. All tools accept a language parameter for language-specific processing.

Line	Text	Syll.	Pattern
1	The snow glows white	4	[1,1,1,1]
2	On the mountain tonight	6	[1,1,2,2]
3	Not a footprint to be seen	7	[1,1,2,1,1,1]
4	A kingdom of isolation	8	[1,2,1,4]

Rhyme scheme: *ABCB* (*tonight/seen* do not rhyme; detected via IPA ending comparison)

Table 2: Example constraint extraction from English source lyrics. Syllable counts are computed via IPA vowel nuclei; patterns via space-based word segmentation.

tion. DETECTRHYME analyzes the IPA endings of a set of lines and returns the rhyme scheme (e.g., “AABB”), ensuring judgments are phonetic rather than orthographic. ANALYZEPATTERN compares a target syllable pattern against the current one, returning the similarity score (Equation 2), per-word differences, and natural-language suggestions for improvement. All tools are registered as callable functions via modern LLM function-calling APIs.

4.3 Constraint Extraction

Before translation begins, the system extracts all three constraints from the source lyrics L_s in language λ_s : (1) **syllable counts** $\mathbf{s} = (s_1, \dots, s_n)$ via COUNTSYLLABLES on each line, (2) **rhyme scheme** R_s via DETECTRHYME on all lines, and (3) **syllable patterns** $\mathbf{P}_s = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ via ANALYZEPATTERN. These constraints are passed to the agent as part of the translation prompt, providing explicit targets for each line. Table 2 shows an example.

4.4 Multi-Phase Agent Loop

Translation proceeds through three phases, implemented as a directed acyclic graph where each node represents a processing step and conditional edges

encode control flow. The design reflects the priority ordering identified by Low (2005): semantic content and rhythm (syllable counts) take precedence, followed by phrasing (patterns), with rhyme considered throughout but given lower priority.

Phase 1: Initial Translation with Tool-Guided Reasoning. The agent receives a structured prompt containing the source lyrics, target language, and all extracted constraints. It generates an initial translation of all lines, following a ReAct loop (Yao et al., 2023): the LLM generates reasoning about how to satisfy constraints, calls tools to verify its output, observes the results, and adjusts within the same generation turn. The prompt instructs the agent to consider all three constraints simultaneously, producing a “best-effort” initial translation.

A typical interaction proceeds as: the agent reasons about the target syllable count for a line, drafts a translation, calls COUNTSYLLABLES to verify, observes the result, and adjusts if needed—continuing until all lines are translated.

Phase 2: Line-by-Line Syllable Refinement. The initial translation rarely satisfies all syllable constraints exactly. In this phase, the agent processes each line individually, computing the actual syllable count and comparing it to the target. If there is a mismatch, the agent receives explicit feedback (e.g., “Line i has s'_i syllables but the target is s_i ; please revise while preserving meaning”) and generates a revised translation. The system re-checks via COUNTSYLLABLES, continuing for up to 10 iterations; if no exact match is found, the closest attempt is retained. This phase is critical because even a single syllable mismatch can make a line unsingable.

Phase 3: Syllable Pattern Refinement. After syllable counts are matched, the agent refines word-level syllable distributions. For each line, ANALYZEPATTERN compares the current pattern against the target and returns structured feedback including the similarity score, per-word differences, and suggestions (e.g., “split the first word into shorter components”). For each mismatched line, the agent generates a revised translation; the revision is accepted only if it improves pattern similarity while maintaining the correct total syllable count, and is otherwise reverted. Lines whose patterns already match the target are left unchanged.

Priority and termination. The three-phase design addresses constraints in order of importance: syllable counts first (hardest constraint), then syllable patterns. Rhyme is monitored throughout Phases 1 and 2 via DETECTRHYME, but is not given a dedicated refinement phase because rhyme violations are generally less disruptive to singability than syllable mismatches (Low, 2005). The process terminates after Phase 3 with a final validation step.

4.5 Worked Example

To illustrate, consider translating “*Not a footprint to be seen*” (7 syllables, pattern [1, 1, 2, 1, 1, 1]) into Chinese. In Phase 1, the agent produces a 7-character translation; syllable counting confirms the match, so Phase 2 skips this line. In Phase 3, pattern analysis reveals the Chinese tokenization yields [3, 2, 2]—similarity 0.38 against the target—and suggests splitting the first word. The agent revises to a translation with single-character tokenization, yielding [1, 1, 1, 1, 1, 1, 1] and similarity 0.71: an improvement, though not exact. This illustrates a fundamental challenge—Chinese word boundaries are determined by semantics and do not always permit arbitrary decomposition.

5 Evaluation Metrics

Standard MT metrics such as BLEU measure n-gram overlap and do not capture whether a translation preserves musical structure. Kim et al. (2023) proposed a computational evaluation framework for singable lyric translation; building on this direction, we propose three complementary metrics, each targeting one of the three musical constraints.

5.1 Syllable Error Rate (SER)

SER measures the overall alignment between the source and translated syllable count sequences using the Levenshtein edit distance (Levenshtein, 1966):

$$\text{SER} = \frac{\text{Lev}(\mathbf{s}, \mathbf{s}')}{\max(|\mathbf{s}|, |\mathbf{s}'|)} \quad (3)$$

where $\mathbf{s} = (s_1, \dots, s_n)$ and $\mathbf{s}' = (s'_1, \dots, s'_m)$ are the source and translated syllable count sequences, and Lev operates over sequences of integers (with substitution cost equal to 1 regardless of the magnitude of the difference).

SER captures both *substitution errors* (a line has the wrong syllable count) and *structural errors* (lines are missing or added). SER = 0 indicates

perfect preservation; higher values indicate greater divergence. Unlike per-line metrics, SER penalizes translations that drop or add lines, which is important because line structure directly corresponds to melodic phrases.

5.2 Syllable Count Relative Error (SCRE)

SCRE provides a normalized, per-line view of syllable accuracy, assuming the translation preserves the number of lines ($m = n$):

$$\text{SCRE} = \frac{1}{n} \sum_{i=1}^n \frac{|s_i - s'_i|}{s_i} \quad (4)$$

SCRE weights errors relative to line length: a two-syllable error on a four-syllable line ($\text{SCRE}_i = 0.5$) is penalized more heavily than on a twelve-syllable line ($\text{SCRE}_i = 0.17$). This reflects the intuition that short lines leave less room for syllabic deviation—a single extra syllable in a three-syllable phrase is far more noticeable than in a long verse. $\text{SCRE} = 0$ indicates all lines match exactly.

SER and SCRE are complementary: SER captures structural alignment including line count mismatches, while SCRE provides percentage-based accuracy that is more interpretable for line-by-line comparison.

5.3 Adjusted Rand Index (ARI)

To measure rhyme scheme preservation, we treat the rhyme scheme as a *clustering* of lines and compute the Adjusted Rand Index (Hubert and Arabie, 1985) between the source and translated clusterings.

Let $R_s = (r_1, \dots, r_n)$ and $R_t = (r'_1, \dots, r'_n)$ be the rhyme scheme labelings. The Rand Index (RI) counts the fraction of line pairs that are either in the same cluster in both schemes or in different clusters in both schemes. ARI adjusts RI for the expected value under random label assignments:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \quad (5)$$

$\text{ARI} \in [-1, 1]$: 1 indicates the translation perfectly preserves the rhyme structure, 0 indicates chance-level agreement, and negative values indicate systematic disagreement. Crucially, ARI is robust to label permutation—a source with scheme *AABB* and a translation with *BBAA* receives $\text{ARI} = 1$, since the grouping structure is identical.

Together, SER, SCRE, and ARI cover the structural dimensions of singable translation. They deliberately do not measure semantic fidelity or naturalness; we view them as complementary to standard MT metrics such as BLEU and COMET. A good singable translation should score well on both semantic quality metrics and structural quality metrics—high BLEU with high SER would indicate a good translation that is unsingable, while low SER with low BLEU would be structurally correct but semantically wrong.

6 Experiments

We conduct preliminary experiments to evaluate the effectiveness of IPA-augmented tool use for constraint-preserving lyrics translation.

6.1 Setup

We evaluate on English-to-Chinese ($\text{en} \rightarrow \text{cmn}$) song translation using 30 test cases (5 lines each) from the publicly available test split of the parallel lyric corpus released by Ou et al. (2023a).² The dataset covers pop, ballad, rock, and musical-theatre genres. Our open-source release (Apache-2.0) includes the full translation framework, IPA tool suite, LangGraph agent definitions, and evaluation scripts as a PyPI package (`blt-toolkit`), enabling reproduction on any user-supplied lyrics in any language pair supported by eSpeak-NG.

Implementation. The agent framework is implemented using LangGraph with conditional edges encoding the control flow between constraint extraction, initial translation, syllable refinement, and pattern refinement. For LLM inference, we use Qwen3-30B (4-bit quantized) served locally via Ollama. The IPA tools (§4.2) are registered as callable functions through LangGraph’s tool-binding interface.

Conditions. We conduct a phase ablation study alongside a supervised baseline:

- **Phase 1 only:** Initial translation with IPA tool access but no iterative refinement.
- **Phase 1+2:** Adds line-by-line syllable count refinement (up to 10 iterations per line).
- **Phase 1+2+3** (full BLT Agent): Adds syllable pattern refinement after syllable counts are matched.

²<https://huggingface.co/datasets/LongshenOu/lyric-trans-en2zh-data>

Method	SER ↓	SCORE ↓	ARI ↑
Phase 1 only	0.7926	0.1949	0.4128
Phase 1+2	<u>0.2000</u>	<u>0.0242</u>	0.2723
Phase 1+2+3 (full)	0.2267	0.0313	<u>0.3290</u>
CLT (Ou et al., 2023a)	0.0467	0.0042	0.0012

Table 3: Phase ablation and baseline comparison (en → cmn, $n=30$). Bold = best overall; underline = best among BLT variants.

- **CLT (Ou et al., 2023a)**: A supervised mBART model fine-tuned with explicit length, rhyme, and word-boundary constraint tokens. This represents a task-specific training approach and serves as an upper bound for constraint satisfaction.

All BLT conditions use the same Qwen3-30B model; CLT uses its own fine-tuned checkpoint. We report SER, SCORE, and ARI as defined in §5.

6.2 Results and Phase Ablation

Table 3 presents the results of the phase ablation study.

Phase 2 is the critical stage. Adding syllable refinement reduces SER by 74.8% ($0.79 \rightarrow 0.20$) and SCORE by 87.6% ($0.19 \rightarrow 0.02$), confirming that iterative tool-verified refinement—not merely tool access during initial generation—drives constraint satisfaction. The cost is a $4.4\times$ increase in inference time ($6.7\text{ s} \rightarrow 29.3\text{ s}$ per test case).

Phase 3 trades syllable accuracy for rhyme. Pattern refinement slightly increases SER (+13%) and SCORE (+29%) because adjusting word boundaries occasionally disrupts syllable counts that Phase 2 had matched. However, ARI improves by 20.8% ($0.27 \rightarrow 0.33$), indicating that pattern-level adjustments help restore rhyme structure. This suggests Phase 3 is most valuable when rhyme preservation is a priority.

Comparison with CLT. The supervised CLT baseline achieves near-perfect syllable accuracy (SCORE 0.004) through explicit constraint tokens and task-specific fine-tuning. However, its ARI is near zero (0.001), indicating that its constraint mechanism does not preserve rhyme structure. The full BLT Agent achieves substantially higher ARI (0.33 vs. 0.001) while requiring no task-specific training—only IPA tools and a general-purpose LLM. CLT is also limited to the single language pair it was trained on, whereas BLT operates on any

Line	Text	Syll.	Tgt
Source (English):			
1	The snow glows white	4	—
2	On the mountain tonight	6	—
3	Not a footprint to be seen	7	—
4	A kingdom of isolation	8	—
Base LLM (Chinese, no tools):			
1	白雪在夜裡閃耀	7	$4\times$
2	今夜在那座山上	7	$6\times$
3	看不見任何一個腳印	9	$7\times$
4	一個與世隔絕的王國	9	$8\times$
BLT Agent (Chinese, with IPA tools):			
1	皚皚白雪	4	$4\checkmark$
2	籠罩今夜山巔	6	$6\checkmark$
3	看不到一絲足跡	7	$7\checkmark$
4	這是孤獨的國度	7	$8\times$

Table 4: Qualitative example: English-to-Chinese translation of the opening lines of *Let It Go*. “Syll.” is the actual character count; “Tgt” is the source syllable count that the translation should match. The BLT Agent matches 3/4 targets; the Base LLM matches 0/4.

language pair supported by Phonemizer’s eSpeak-NG backend, covering over 100 languages without retraining.

6.3 Qualitative Analysis

Table 4 illustrates the pipeline on the *Let It Go* excerpt from Table 2, translated from English to Chinese. The Base LLM produces natural but unconstrained translations: line counts range from 7 to 9 characters, overshooting every target. The BLT Agent, guided by COUNTSYLLABLES verification in Phase 2, matches three of four targets exactly. Line 4 (target 8) remains one syllable short at 7—Phase 2 exhausted its 10 refinement attempts without finding an 8-character alternative that preserved meaning, retaining the closest attempt. Phase 3 pattern refinement has limited effect here because Chinese character-level tokenization produces uniform $[1, 1, \dots]$ patterns regardless of word grouping.

6.4 Discussion and Future Directions

Training-free versus supervised. The CLT comparison highlights a fundamental trade-off: supervised models achieve superior syllable accuracy but require parallel lyric corpora and per-language-pair training, whereas the BLT Agent operates on any language pair without retraining. Since Phonemizer supports over 100 languages via eSpeak-NG, and the agent loop is language-agnostic, BLT can translate between any source–target pair (e.g., Japanese–French, Korean–Spanish) by simply specifying the

language codes. This makes the framework immediately applicable to low-resource language pairs where no parallel lyric data exists.

Phase 3 and the accuracy–rhyme trade-off.

Phase 3 improves ARI at the cost of slight SER/SCRE degradation, suggesting that pattern refinement should be applied selectively—e.g., only when the source has a clear rhyme structure (at least two rhyme classes with two or more lines each). Adding a dedicated rhyme refinement phase that directly optimizes ARI is a natural next step.

Broader evaluation. Our evaluation uses a single language pair and LLM (Qwen3-30B). Extending to typologically diverse pairs, multiple model sizes, and human judgments of singability and semantic fidelity would strengthen generalizability claims. The MAVL benchmark (Cho et al., 2025) may facilitate multilingual evaluation.

7 Conclusion

We have presented a framework for lyrics translation that preserves musical constraints through tool-augmented LLM agents. We formalized three musical constraints—syllable counts, rhyme schemes, and syllable patterns—and described IPA-based methods for their extraction and verification. By equipping an LLM with phonetic analysis tools and structuring translation as a multi-phase ReAct loop, our system iteratively refines translations to satisfy these constraints in priority order: syllable counts first, then patterns, with rhyme monitored throughout. The proposed metrics—SER, SCRE, and ARI—provide automatic measurement of constraint preservation that complements standard translation quality evaluation.

A phase ablation study on English-to-Chinese translation shows that iterative syllable refinement (Phase 2) accounts for the majority of improvement (87.6% SCRE reduction), while comparison with a supervised baseline (Ou et al., 2023a) reveals a complementary strength: the BLT Agent preserves rhyme structure (ARI 0.33 vs. 0.001) without task-specific training. Unlike constrained decoding (Hokamp and Liu, 2017; Post and Vilar, 2018) or supervised approaches that require per-language-pair training, our framework is language-agnostic: any LLM with function-calling capabilities can serve as the agent, and the IPA tools support over 100 languages via eSpeak-NG, enabling translation between arbitrary language pairs with-

out retraining.

Future work will focus on refining the agent loop to close the accuracy gap with supervised methods, experimenting with larger or more capable LLMs, broader language evaluation, human judgments of singability, and integration with singing voice synthesis (Ren et al., 2020) for end-to-end singable translation.

Limitations

Our approach has several limitations. First, the framework’s correctness depends on Phonemizer / eSpeak-NG for IPA transcription, which has uneven quality across languages; we do not provide an intrinsic evaluation of syllable-counting or rhyme-detection accuracy. Second, Phase 3 (pattern refinement) can degrade syllable accuracy that Phase 2 achieved, and the framework lacks a dedicated rhyme refinement phase. Third, our evaluation covers only English-to-Chinese ($n=30$) with a single LLM; although the framework is language-agnostic in principle, we have not validated it on other language pairs. Fourth, the supervised CLT baseline substantially outperforms BLT on syllable metrics, indicating that the agent approach has room to improve on constraint satisfaction—the current advantage is in rhyme preservation and language-pair flexibility rather than raw accuracy. Fifth, the multi-phase refinement loop introduces latency (29–30 s vs. 0.4 s for CLT per test case), which may limit practical deployment. Sixth, our metrics measure structural constraint preservation but not semantic fidelity or naturalness; a complete evaluation requires human judgments. Finally, the technical approach—calling deterministic phonetic tools inside a ReAct loop—applies established patterns (Yao et al., 2023; Gao et al., 2023; Schick et al., 2023) to a new domain; the primary contribution is the formalization of musical constraints and the IPA-based tool suite rather than a novel agent architecture.

References

- Mathieu Bernard and Hadrien Titeux. 2021. [Phonemizer: Text to phones transcription for multiple languages in Python](#). *Journal of Open Source Software*, 6(68):3958.
- Yiwen Chen and Simone Teufel. 2024. Scansion-based lyrics generation. In *Proceedings of the 2024 Joint International Conference on Computational Linguis-*

666	tics, <i>Language Resources and Evaluation (LREC-</i>	Jey Han Lau, Trevor Cohn, Timothy Baldwin, Julian	718
667	<i>COLING 2024</i>), pages 14370–14381.	Brooke, and Adam Hammond. 2018. Deep-speare:	719
668	Woohyun Cho, Youngmin Kim, Sunghyun Lee, and	A joint neural model of poetic language, meter and	720
669	Youngjae Yu. 2025. MAVL: A multilingual audio-	rhyme . In <i>Proceedings of the 56th Annual Meet-</i>	721
670	video lyrics dataset for animated song translation. In	<i>ing of the Association for Computational Linguistics</i>	722
671	<i>Proceedings of the 2025 Conference on Empirical</i>	<i>(Volume 1: Long Papers)</i> , pages 1948–1958.	723
672	<i>Methods in Natural Language Processing</i> .	Vladimir I. Levenshtein. 1966. Binary codes capable of	724
673	Johan Franzon. 2008. Choices in song translation:	correcting deletions, insertions, and reversals. <i>Soviet</i>	725
674	Singability in print, subtitles and sung performance.	<i>Physics Doklady</i> , 10(8):707–710.	726
675	<i>The Translator</i> , 14(2):373–399.	Qihao Liang, Xichu Ma, Finale Doshi-Velez, Brian Lim,	727
676	Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon,	and Ye Wang. 2024. XAI-lyricist: Improving the	728
677	Pengfei Liu, Yiming Yang, Jamie Callan, and Gra-	singability of AI-generated lyrics with prosody expla-	729
678	ham Neubig. 2023. PAL: Program-aided language	nations. In <i>Proceedings of the 33rd International</i>	730
679	models . In <i>Proceedings of the 40th International</i>	<i>Joint Conference on Artificial Intelligence</i> , pages	731
680	<i>Conference on Machine Learning</i> , pages 10764–	7877–7885.	732
681	10799.	Peter Low. 2005. The pentathlon approach to trans-	733
682	Marjan Ghazvininejad, Yejin Choi, and Kevin Knight.	lating songs. In Dinda L. Gorlé, editor, <i>Song and</i>	734
683	2018. Neural poetry translation. In <i>Proceedings of</i>	<i>Significance: Virtues and Vices of Vocal Translation</i> ,	735
684	<i>the 2018 Conference of the North American Chap-</i>	pages 185–212. Rodopi, Amsterdam.	736
685	<i>ter of the Association for Computational Linguistics:</i>	Peter Low. 2022. Translating the texts of songs and	737
686	<i>Human Language Technologies, Volume 2 (Short Pa-</i>	other vocal music . In Kirsten Malmkjær, editor,	738
687	<i>pers</i>).	<i>The Cambridge Handbook of Translation</i> . Cambridge	739
688	Marjan Ghazvininejad, Xing Shi, Yejin Choi, and Kevin	University Press.	740
689	Knight. 2016. Generating topical poetry . In <i>Proceed-</i>	David R. Mortensen, Patrick Littell, Akash Bharadwaj,	741
690	<i>ings of the 2016 Conference on Empirical Methods</i>	Kartik Goyal, Chris Dyer, and Lori Levin. 2016. Pan-	742
691	<i>in Natural Language Processing</i> , pages 1183–1191.	Phon: A resource for mapping IPA segments to artic-	743
692	Fenfei Guo, Chen Zhang, Zhirui Zhang, Qixin He,	ulatory feature vectors. In <i>Proceedings of COLING</i>	744
693	Kecheng Zhang, Jun Xie, and Jordan Boyd-Graber.	<i>2016, the 26th International Conference on Computa-</i>	745
694	2022. Automatic song translation for tonal languages.	<i>tional Linguistics: Technical Papers</i> , pages 3475–	746
695	In <i>Findings of the Association for Computational Lin-</i>	3484.	747
696	<i>guistics: ACL 2022</i> .	Longshen Ou, Xichu Ma, Min-Yen Kan, and Ye Wang.	748
697	Chris Hokamp and Qun Liu. 2017. Lexically con-	2023a. Songs across borders: Singable and control-	749
698	strained decoding for sequence generation using grid	lable neural lyric translation . In <i>Proceedings of the</i>	750
699	beam search . In <i>Proceedings of the 55th Annual</i>	<i>61st Annual Meeting of the Association for Computa-</i>	751
700	<i>Meeting of the Association for Computational Lin-</i>	<i>tional Linguistics (Volume 1: Long Papers)</i> , pages	752
701	<i>guistics (Volume 1: Long Papers)</i> , pages 1535–1546.	447–467.	753
702	Jack Hopkins and Douwe Kiela. 2017. Automatically	Longshen Ou, Xichu Ma, and Ye Wang. 2023b. LOAF-	754
703	generating rhythmic verse with neural networks . In	M2L: Joint learning of wording and formatting for	755
704	<i>Proceedings of the 55th Annual Meeting of the As-</i>	singable melody-to-lyric generation. <i>arXiv preprint</i>	756
705	<i>sociation for Computational Linguistics (Volume 1:</i>	<i>arXiv:2307.02146</i> .	757
706	<i>Long Papers)</i> , pages 168–178.	Matt Post and David Vilar. 2018. Fast lexically con-	758
707	Lawrence Hubert and Phipps Arabie. 1985. Comparing	strained decoding with dynamic beam allocation for	759
708	partitions . <i>Journal of Classification</i> , 2(1):193–218.	neural machine translation . In <i>Proceedings of the</i>	760
709	Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Xing	<i>2018 Conference of the North American Chapter of</i>	761
710	Wang, Shuming Shi, and Zhaopeng Tu. 2023. Is	<i>the Association for Computational Linguistics: Hu-</i>	762
711	ChatGPT a good translator? Yes with GPT-4 as the	<i>man Language Technologies, Volume 1 (Long Pa-</i>	763
712	engine. <i>arXiv preprint arXiv:2301.08745</i> .	<i>pers)</i> , pages 1314–1324.	764
713	Haven Kim, Kento Watanabe, Masataka Goto, and	Le Ren, Xiangjian Zeng, Qingqiang Wu, and Ruox-	765
714	Juhan Nam. 2023. A computational evaluation frame-	uan Liang. 2025. LyriCAR: A difficulty-aware	766
715	work for singable lyric translation. In <i>Proceedings of</i>	curriculum reinforcement learning framework for	767
716	<i>the 24th International Society for Music Information</i>	controllable lyric translation. In <i>arXiv preprint</i>	768
717	<i>Retrieval Conference</i> .	<i>arXiv:2510.19967</i> .	769
		Yi Ren, Xu Tan, Tao Qin, Jian Luan, Zhou Zhao, and	770
		Tie-Yan Liu. 2020. DeepSinger: Singing voice syn-	771
		thesis with data mined from the web . In <i>Proceedings</i>	772
		<i>of the 26th ACM SIGKDD International Conference</i>	773

on *Knowledge Discovery and Data Mining*, pages 1979–1989.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *Proceedings of the Eleventh International Conference on Learning Representations*.

Zhuorui Ye, Jinhan Li, and Rongwu Xu. 2024. [Sing it, narrate it: Quality musical lyrics translation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Suhyeon Yoo, Khai N. Truong, and Young-Ho Kim. 2025. ELMI: Interactive and intelligent sign language translation of lyrics for song signing. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*.

Jiaxing Yu, Xinda Wu, Yunfei Xu, Tiejiao Zhang, Songruoyao Wu, Le Ma, and Kejun Zhang. 2025. SongGLM: Lyric-to-melody generation with 2D alignment encoding and multi-task pre-training. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence*.

Songyan Zhao, Bingxuan Li, Yufei Tian, and Nanyun Peng. 2025. REFFLY: Melody-constrained lyrics editing model. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11295–11315.